

Experiment No. 8

Aim: Implementing Deep Learning Applications like Handwritten Digit Recognition System.

Theory: The handwritten digit recognition is the ability of computers to recognize human handwritten digits. It is a hard task for the machine because handwritten digits are not perfect and can be made with many different flavors. The handwritten digit recognition is the solution to this problem which uses the image of a digit and recognizes the digit present in the image.

Created the Dataset Using the 100 images each for particular number using the captured images And formatted in the (28, 28, 1) pixel. Understanding the patterns of each number using the pixel by identify the index number of the captured images and drawing in paint to match the pattern of frames

Input:

Step 1: Installing and Importing Necessary Packages

```
!pip install pyscreenshot  
!pip install opencv-python  
!pip install pandas  
!pip install scikit-learn
```

Step 2: Creating Images for Digits (0 – 9)

```
[2]: def one_time():  
      import pyscreenshot as ImageGrab  
      import time  
      images_folder="captured_images/9/"  
      for i in range(0,100):  
          time.sleep(6)  
          im=ImageGrab.grab(bbox=(60,170,400,550)) #x1,y1,x2,y2  
          print("saved.....",i)  
          im.save(images_folder+str(i)+'.png')  
          print("clear screen now and redraw now.....")
```

Step 3: Generate Dataset Using the Captured Images

```
[4]: import cv2
import csv
import glob

header = ["label"]
for i in range(0,784):
    header.append("pixel"+str(i))
with open('dataset.csv', 'a') as f:
    writer = csv.writer(f)
    writer.writerow(header)

for label in range(10):
    dirList = glob.glob("captured_images/"+str(label)+"/*.png")

    for img_path in dirList:
        im= cv2.imread(img_path)
        im_gray = cv2.cvtColor(im,cv2.COLOR_BGR2GRAY)
        im_gray = cv2.GaussianBlur(im_gray,(15,15), 0)
        roi= cv2.resize(im_gray,(28,28), interpolation=cv2.INTER_AREA)

        data=[]
        data.append(label)
        rows, cols = roi.shape
```

```
## Fill the data array with pixels one by one.
for i in range(rows):
    for j in range(cols):
        k =roi[i,j]
        if k>100:
            k=1
        else:
            k=0
        data.append(k)
with open('dataset.csv', 'a') as f:
    writer = csv.writer(f)
    writer.writerow(data)
```

Step 4: Load the Dataset

```
[5]: import pandas as pd
      from sklearn.utils import shuffle
      data = pd.read_csv('dataset.csv')
      data = shuffle(data)
      print(data)
```

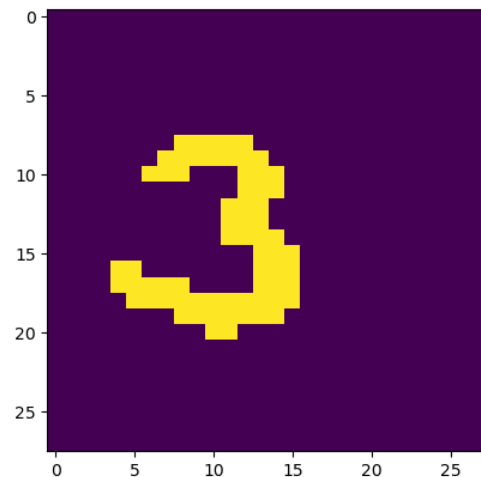
	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	\
2829	8	0	0	0	0	0	0	0	0	0	
1029	0	0	0	0	0	0	0	0	0	0	
320	3	0	0	0	0	0	0	0	0	0	
2370	3	0	0	0	0	0	0	0	0	0	
1117	1	0	0	0	0	0	0	0	0	0	
...	...	...	...	...	...	...	...	...	...	...	
2859	8	0	0	0	0	0	0	0	0	0	
890	8	0	0	0	0	0	0	0	0	0	
635	6	0	0	0	0	0	0	0	0	0	
1504	5	0	0	0	0	0	0	0	0	0	
2132	1	0	0	0	0	0	0	0	0	0	

Step 5: Preview of one image using matplotlib

```
[12]: %matplotlib inline
      import matplotlib.pyplot as plt
      import cv2
      idx = 314
      img = X.loc[idx].values.reshape(28,28)
      print(Y[idx])
      plt.imshow(img)
```

Matplotlib is building the font cache; this may take a moment.

3



Step 6: Train – Test Split

```
[14]: from sklearn.model_selection import train_test_split
      train_x, test_x, train_y, test_y = train_test_split(X, Y, test_size = 0.2)
```

Step 7: Fit the model using svc and also save the model using joblib

```
[16]: import joblib
      from sklearn.svm import SVC
      classifier=SVC(kernel="linear", random_state=6)
      classifier.fit(train_x, train_y)
      joblib.dump(classifier, "model/digit_recognizer")
```

```
[16]: ['model/digit_recognizer']
```

Step 8: Calculate Accuracy

```
[18]: from sklearn import metrics
      prediction=classifier.predict(test_x)
      print("Accuracy= ", metrics.accuracy_score(prediction, test_y))

      Accuracy= 1.0
```

Step 9: prediction of image drawn in paint

```
*[20]: import joblib
      import cv2
      import numpy as np
      import time
      import pyscreenshot as ImageGrab

      model=joblib.load("model/digit_recognizer")
      images_folder="img/"

      while True:
          img=ImageGrab.grab(bbox=(60,170,400,500))

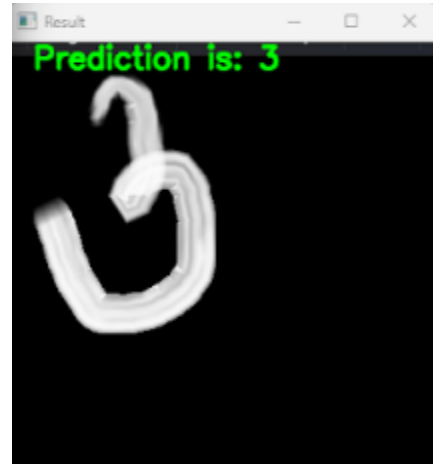
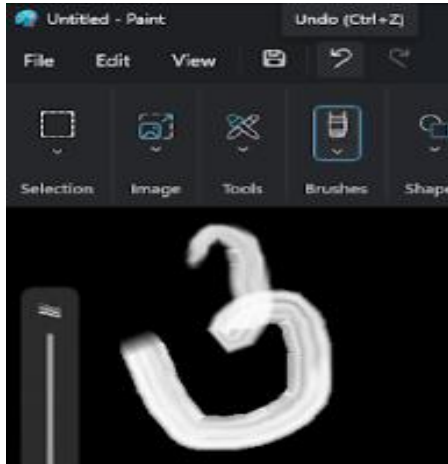
          img.save(images_folder+"img.png")
          im = cv2.imread(images_folder+"img.png")
          im_gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
          im_gray = cv2.GaussianBlur(im_gray, (15,15), 0)

          #Threshold the image
          ret, im_th = cv2.threshold(im_gray, 100, 255, cv2.THRESH_BINARY)
          roi = cv2.resize(im_th, (28,28), interpolation = cv2.INTER_AREA)

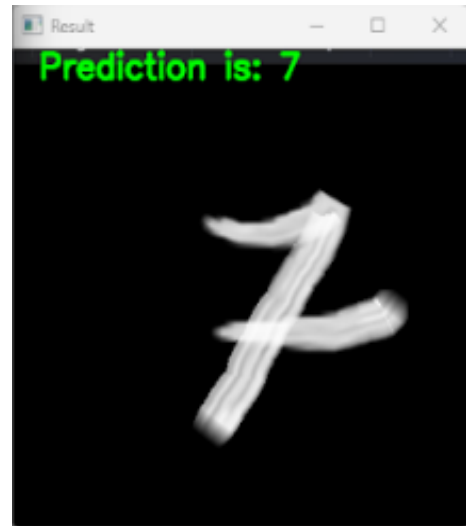
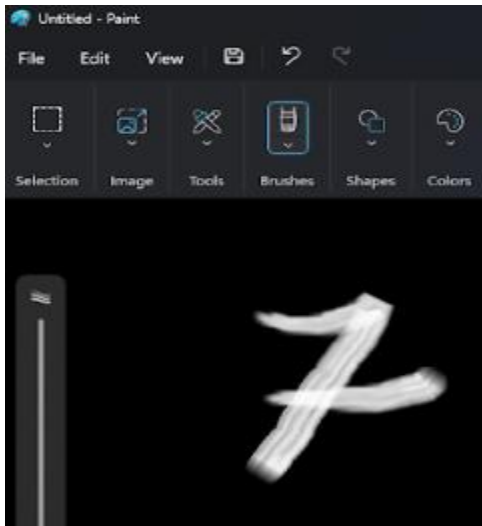
          rows, cols=roi.shape

          X = []
```

Prediction: 3



Prediction: 7



Conclusion: Thus, we have successfully studied and implemented Handwritten Digit Recognition System by creating original dataset.

